

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.Doi Number

Deep Learning based Improved Classification System for Designing Tomato Harvesting Robot

Li Zhang^{1,2}, Jingdun Jia³, Guan Gui⁴, Senior Member, IEEE, Xia Hao^{1,2}, Wanlin Gao^{1,2} and Minjuan Wang^{1,2}

¹College of Information and Electrical Engineering, China Agricultural University, Beijing, 100083China

²Key Laboratory of Agricultural Informatization Standardization, Ministry of Agriculture, Beijing, 100083China

³National Rural Technology Development Center, Ministry of Science and Technology, Beijing, 100862China

⁴Nanjing University of Posts and Telecommunications, Nanjing, 210003China

Corresponding author: Wanlin Gao (e-mail: wanlin_cau@163.com), Minjuan Wang (e-mail: minjuan@cau.edu.cn)

This work was supported by Project of Scientific Operating Expenses, Ministry of Education of China (2017PT19) and China Postdoctoral Science Foundation (2018M630222). All the mentioned support is gratefully acknowledged.

ABSTRACT Maturity level based classification system plays an essential role in the design of tomato harvesting robot. Traditional knowledge based systems are unable to meet the current production management requirements of precision picking, because they are time-consuming and have low accuracy. Our research proposes an improved deep learning based classification method that improves accuracy and scalability of tomato ripeness with a small amount of training data. This study was on the relationship of different dataset augmentation methods and prediction results of final classification task. We implemented classification systems based on convolutional neural network (CNN), by training and validating the model on different augmented datasets and tried to choose an optimal augmentation method for datasets. The experimental results showed an average accuracy of 91.9% with a less than 0.01 second prediction time. Compared to existing methods, our solution achieved better prediction results both in terms of accuracy and time consumption. Moreover, this is a versatile method and can be extended to other related fields.

INDEX TERMS convolutional neural network, classification, data augmentation, tomato harvesting robot, deep learning

I. INTRODUCTION

Tomato is one of the most popular vegetables in human daily life since it is consumed by millions every day. However, with the trend of aging work force, the labor cost is rising and it has become one of the limiting factors in many agricultural industries. On the one hand, a large number of agricultural enterprises are facing the challenges of low profit. While, on the other hand, with the growing world population, the production of tomato still needs to satisfy the demand. The harvesting robot of tomato seems to be a plausible way to solve these critical issues of keeping tomato quality control with reduced labor cost. Due to these reasons, many researchers have been working on developing robots for fruit and vegetable harvesting for the last few decades [1], [2].

The color of tomato is a major index to judge maturity. Tomato fruit passes through five different stages of maturity. These can be recognized through color changes from green turning to light pink, pink, light red, and then red, which classify them into five distinct categories. An

appropriate appearance of the produce brings high price for the company. So, one must take into account the length of transport route and storage time for an optimal harvest. In general, from green color tomato needs 21 to 28 days for breakers, 15 to 20 days for the turning, 7 to 14 days for pink, 5 to 6 days for light red, 2 to 4 days for red stages [3]. Therefore, it is an important task to improve the tomato classification system for the design of harvesting robot.

In recent years, the method based on machine vision and pattern recognition has been well studied and applied, especially in many intelligent agricultural products' processing or sorting [4], [5], [6]. Specifically, computer vision is one of the most important parts of the harvesting robot. However, the methods based on machine vision are selected by experienced personnel. Obviously, such methods have drawbacks in flexibility and timeliness that make them hard to apply in farm enterprises. Furthermore, the development of such system with good performance in terms of accuracy timeliness and scalability should resolve many challenging issues. These include tasks such as illumination

variation, occlusions and so on for conducting work on various factors. Although many researchers have used machine vision technologies, there still has a long distance to supply for the automatic harvesting robot. Both accuracy and efficiency have not been achieved for designing such robot.

Recently, convolutional neural network (CNN) based classification systems have made ground breaking advances in many tasks. Deep neural network (DNN) system often encounters over-fitting problems although it has shown outstanding performance in many aspects. The problems of over-fitting are mainly caused by three reasons, i.e. complex models, data noise and limited training data [7]. To create a dataset with enough samples is often a difficult and time-consuming task. Particularly, some images are hard to obtain, such as specific disease of one kind of plant. Therefore, data augmentation is an effective way to pursue, by artificially increasing training data, when the number of images in the dataset is insufficient.

Our motivation for this study was to look for an efficient procedure for observing tomato ripeness. Moreover, we needed to find a method having a good performance both in predicting time and accuracy. Furthermore, we aimed to design the classification system with extensible capabilities, so that it can be applied to harvesting robot commendably. Based on the above considerations, we designed and implemented classification system as shown in Fig. 1.

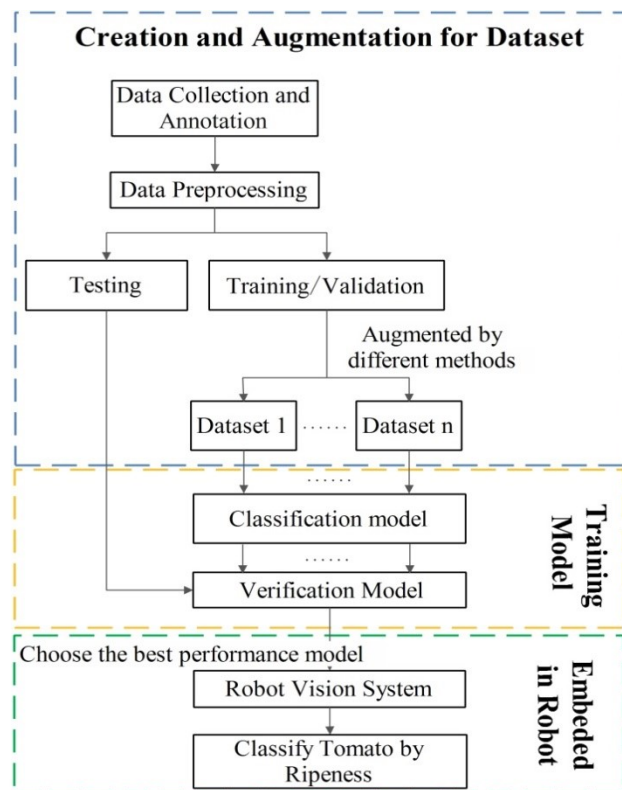


FIGURE 1. The flow chart of the classification system for harvesting robot.

We analyzed the relationship between tomato storage duration and changes in appearance, and then divided images of tomato into five categories according to ripeness in this study. We propose novel network model architecture with less complexity to implement the task of fast classification. For the dataset construction, we used several ways to collect the images of tomato, which include different ripeness levels. After labeling each image, we verified the accuracy of the dataset. Considering that the acquisition of data sets is a time-consuming work, we took advantage of several different methods for dataset augmentation. By comparing both the performance of training and prediction results of the model under different augmentation methods, we derived the most suitable augmentation method for this study. This method offers suggestion for designing tomato harvesting robot.

The structure of this paper is as follows. In Section I, we introduce the motivation of this research, and also some relevant background. In Section II, we show the establishment of our dataset, mainly including the collection of images, annotation, filtering of inappropriate data, and data augmentation. We build a novel framework for ripeness classification based on deep learning in Section III. Then, we show some results of the classification system on different datasets in Section IV. In Section V, we discuss the conclusions of this research.

II. RELATED WORKS AND BACKGROUND

Estimation of tomato maturity is a significant and important study for automatic picking. To estimate the maturity of tomatoes, Goel *et al.* proposed a color based method, where the ripeness classification system achieved high accuracy [8]. Pavithra *et al.* used machine learning technology for automatic detection and sorting of cherry tomatoes [9] and they designed classifier system to improve accuracy and economize time consumption. Lu *et al.* used machine vision and Visible/Near-Infrared Spectroscopy technologies to comprehend rapid assessment of tomato ripeness [10]. Mohapatra *et al.* adopted image processing approach for red banana's ripening grade determination [11]. Although these methods gave good performance through experiments in certain environments, they are still difficult to apply.

At present, the classification system based on CNN has achieved good results in many areas. In this regard, researchers have proposed different data augmentation methods. For example, Zhou *et al.* proposed cross-label suppression dictionary learning for signal representation in face recognition to preserve the label property effectively [12]. Chen *et al.* proposed a novel approach that applies cascades of three deep convolutional neural networks (DCNNs) methods to detect the defect of fasteners [13]. Li *et al.* proposed FingerNet, which consists of one common convolution part and two different de-convolution parts to enhance fingerprint [14]. To effectively suppress the outliers and accurately reconstruct the image from compressive measured data, Li *et al.* presented a novel multiplier network

based algorithm to achieve better performance in image reconstruction [15]. Ma *et al.* presented a new method based on variational Bayesian learning method, and achieved flexible performance for modeling vector with positive elements on Dirichlet process mixture of the inverted Dirichlet distributions [16]. Also, there are some CNN-based studies for face recognition [17], wireless communications [18] [19] [20], automatic speaker verification [21] and internet of things [22]. Similarly, driven by the remarkable success of deep learning, CNN-based classification or identification systems have recently made ground breaking advances in agricultural industry. For example, investigations have been done on classification and identification system for crop diseases [23], [24]. Also many researchers have developed systems for plant identification and detection [25], [26].

Over-fitting is one of the most serious problems based on the CNN method. Beneficial to the effective way of preventing over-fitting problems, many deep learning-based studies exploit data augmentation methods. For example, a new approach based on CNN for alcoholism detection task with data augmentation methods only uses one hundred training image [27]. For road detection, Bulnes *et al.* used two ways for dataset augmentation. First is a geometric transformation, which includes random affine transformations, perspective transformations, mirroring and so on. The second is called pixel value changes, which includes noise, blur and color changes. The final experimental results showed that training on data augmentation improved performance by 1 to 2% [28]. By random rotation or adding several kinds of noise separately, Hussein *et al.* augmented CT images for nodule characterization of lung. They compared the identification accuracy on different datasets to prove the superiority of the proposed method [29]. Ma *et al.* proposed another novel method for bounded support data that can be used in many important applications [30].

III. DATASET

A. IMAGE ANNOTATION AND VERIFICATION

During this study, we took nearly 200 pieces of color images, containing five maturity levels, collected from farm during daytime under natural light conditions. Each maturity level included more than 30 tomatoes' images. This was a relatively small amount of data compared to that needed generally for network training.

According to the market demand, we took both quality and storage life into consideration and classified acquired images into five categories. Table I shows both the expiry date and quality of each stage. By these standards, we divided these 200 images into their corresponding categories.

t-Distributed Stochastic Neighborhood Embedding (t-SNE) is an algorithm derived from Stochastic

Neighborhood Embedding (SNE) [31]. The idea of t-SNE is to view whether high-dimensional data x_i represents points in high-dimensional space. A nonlinear mapping method that is used to map it at low dimensional space y_i .

TABLE I
QUALITY OF APPEARANCE AND EXPIRY TIME FOR EACH CATEGORY

Name	Color	Storage Time (Days)	Sample
LV1	Breakers	21 ~ 28	
LV2	Turning	15 ~ 20	
LV3	Pink	7 ~ 14	
LV4	Light red	5 ~ 6	
LV5	Red	2 ~ 4	

In high dimensional space, the pairwise distance between two points is converted into a joint probabilistic distance p_{ij} . The transformed equation can be formulated as Eq. (1).

$$p_{ij} = \frac{\exp(-\|x_i - x_j\|/2\sigma^2)}{\sum_k \sum_{l \neq k} \exp(-\|x_l - x_k\|/2\sigma^2)}, \text{ for } \forall i, j : i \neq j \quad (1)$$

In a low dimensional space, the pairwise distance between two points is converted into a joint probabilistic distance q_{ij} defined as Eq. (2).

$$q_{ij} = \frac{\left(1 + \|y_i - y_j\|^2\right)^{-1}}{\sum_k \sum_{l \neq k} \left(1 + \|y_l - y_k\|^2\right)^{-1}}, \text{ for } \forall i \forall j : i \neq j \quad (2)$$

By minimizing the Kullback-Leibler divergence measuring, t-SNE gets the low-dimensional represented by the cost function, can be formulated as Eq. (3).

$$C = KL(P\|Q) = \sum_i \sum_{j \neq i} p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (3)$$

Therefore, benefiting from the advantage of t-SNE method, after manual annotation images, we exploit t-SNE method to check the distribution of the Dataset. **Fig. 2** shows the result of a part of dataset. From the result we can find that the image is basically in accordance with the level of maturity gathered together. We also deleted undesirable data base in this result.



FIGURE 2. The visualization of a part of dataset images on t-SNE distribution.

B. AUGMENTATION METHODS

Creating a data set for learning often requires a lot of energy. Collection, data cleaning, tagging, and so on, takes a lot of time. Taking into account the situations of more usage scenario, such as random noising and translation of size, this paper proposes two types of data augmentation operations to alleviate these problems.

1) GEOMETRIC TRANSFORMATIONS

Scaling and rotations are two ways for geometric transformations. In order to find the best augmentation methods, we generated three datasets in this section.

$S(p, q)$, $D(j, k)$ represent the source and target point of the discrete image. These two points present as $s(u_p, v_q)$ and $d(x_j, y_k)$ in Descartes coordinate system. For the scaling transformation, we exploit the following formula, shown as Eq. (4).

$$\begin{cases} x_j = s_x u_p \\ y_k = s_y v_q \end{cases} \quad (4)$$

Where s_x and s_y are random non-negative scaling coefficient on horizontal and vertical axes. We generated the dataset S based on random scaling transformation. For the rotations transformation, we exploit the following formula Eq. (5).

$$\begin{cases} x_j = u_p \cos \theta - v_q \sin \theta \\ y_k = u_p \sin \theta + v_q \cos \theta \end{cases} \quad (5)$$

θ represents the angle between the rotation image and the original image in counter clock wise direction on the horizontal axis. $\theta \in (0, 360^\circ]$. We generated the dataset R based on random rotation transformation.

Then, we generated the datasets R & S based on datasets S and R. The number of each category in each dataset is shown in Fig. 3.

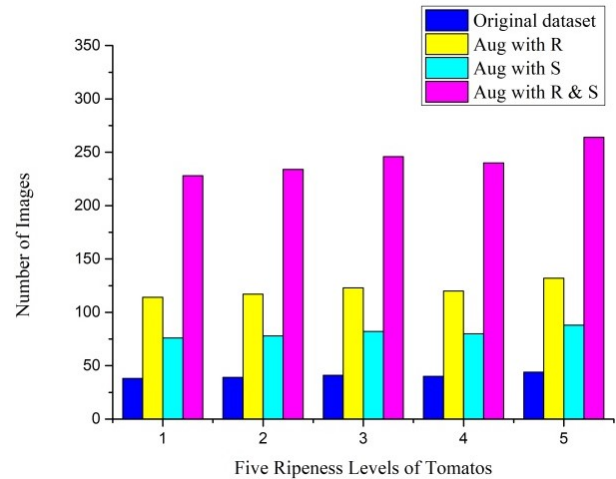


FIGURE 3. The number of images for each category.

2) RANDOM NOISE

We adopted three types of noises, i.e. Pepper, Salt, Gaussian for data augmentation methods. Probability density function (PDF) of Gaussian expression as Eq. (6).

$$p_g(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(z-\bar{z})^2/2\sigma^2} \quad (6)$$

Where z represents the gray scale of image. $\bar{z}$ and σ represent the mean value and standard deviation of z respectively. The PDF of Pepper noise expression as Eq. (7).

$$p_p(z) = \begin{cases} p_p & z = p \\ 0 & \text{else} \end{cases} \quad (7)$$

Where p_p represents probability with pepper noise occurrence. The PDF of Salt noise expression as Eq. (8).

$$p_s(z) = \begin{cases} p_s & z = s \\ 0 & \text{else} \end{cases} \quad (8)$$

Where p_p represents probability with salt noise occurrence. Based on the above probability models, the corresponding random noises were generated respectively.

3) COMBINATION

In order to study the relationship between different ways of data augmentation and the predicted results of this task, we combined the two ways of geometric transformations and random noise by adding Pepper, Salt and Gaussian to the datasets of R, S and R & S separately, and then got nine kinds of datasets for training. They are R & PN, S & PN, S & R & PN, R & SN, S & SN, S & R & SN, R & GN, S & GN, and S & R & GN.

IV. CLASSIFICATION ARCHITECTURE

In this paper, we designed classification architecture shown in **Fig. 4**. The purpose of design classification architecture is to maintain overall information, while preserving local details, with short response time of prediction. Based on the above considerations, we designed this architecture, which consists of three parts. The first part is to input color images of three channels, and these images have 200 pixels attributes of both the height and width. In the second part, we exploited five layers of CNN to extract features. The convolution kernel sizes are 9×9 , 5×5 and 3×3 . In order to retain features, reduce unnecessary parameters and improve the speed of calculation, we also inserted two layers of max-pooling in CNN layers. The last part is for final result classification by a fully connected layer.

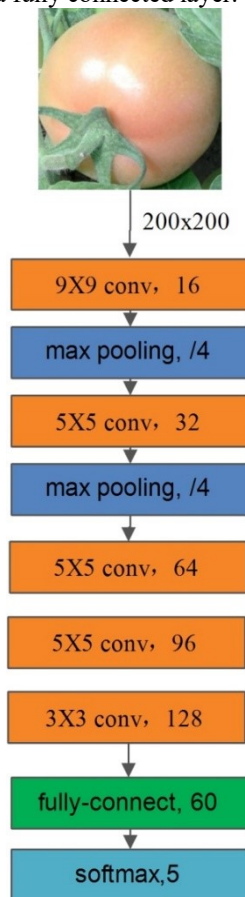


FIGURE 4. CNN based architecture for classification system.

A. FEATURE EXTRACTOR

There are many ways to extract features, such as SVM [32], HOG [33], SIFT [34] and so on. In general, feature extraction based on image processing or machine learning requires the selection of appropriate features according to experience and is not suitable for expansibility of system. In the present study, we exploited CNN based methods to extract features of images without any extra pre-processing. This part mainly includes the following three sub-parts.

1) CONVOLUTIONAL LAYERS

There are five convolutional layers in the design of feature extraction architecture. The kernels of these five convolutional layers are 9×9 with the dimension of 16, 5×5 with the dimension of 32, 64 and 128, and 3×3 with the dimension of 128.

2) ACTIVATION FUNCTION

Several activation functions have been proposed, where ReLU [35] is one of the most popular functions in many classification tasks. The function is defined as Eq. (9).

$$f(x) = \max(0, x) \quad (9)$$

The function of ReLU only needs a threshold to get the activation value without other extra mass operations. Therefore, the network based on such activation function converges faster than most of these. That's why, we exploited ReLU as the activation function of this architecture.

3) POOLING

When a deep learning based architecture gets deeper, it gets larger parameters, more calculations and easy to occur over-fitting phenomenon.

The most important function of pooling layer is to keep invariance in the main feature information, reduce parameters and prevent over-fitting. Commonly, mean-pooling and max-pooling are two forms for the pooling layer. Mean-pooling is calculating the average value of image area as the pooled value of this area.

Similarly, Max-pooling is choosing the max value of image area as the pooled value of this area. In this research, we exploited max-pooling with the size of 4×4 after each convolution layer.

B. CLASSIFIER

Each node of the fully connected layer is connected with all nodes of the previous layer to combine the extracted features with the previous edges. Due to the fully connected characteristics, generally there are more parameters than other layers. In this task, we exploited only one fully connected layer with 32 neurons to connect with the last convolution layer. Softmax [36] [37] model can be used to effectively solve classification problems. The function is given as Eq. (10).

$$p_j = \frac{e^{x_j}}{\sum_{i=1}^K e^{x_i}} \quad (10)$$

Where p_j represents the probability of each category $j \in [1, 5]$ and the value of k is 32. The states of softmax model are mutually constrained, that means only one of the K has value. By exploiting softmax model, we can calculate the probability of each category.

C. TRAINING STRATEGY

We used cross-entropy [38] as the loss function during the model training. The loss function is defined as Eq. (11).

$$C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)] \quad (11)$$

Where y is the expected output value, and a is the actual output. Stochastic gradient descent (SGD) [39] is exploited for each iteration. The expression can be given as Eq. (12).

$$\theta^i = \theta^{i-1} - \alpha \frac{\partial}{\partial \theta^{i-1}} J(\theta^{i-1}) \quad (12)$$

Where α is learning rate, it controls the range of change for every iteration i of a given path toward $J(\theta^{i-1})$, and partial derivative of function $J(\theta^{i-1})$ to the variable θ^{i-1} , represents the direction of maximum change in $J(\theta^{i-1})$. Through the training of the model, we achieved a better performance.

V. EXPERIMENTS

The experiments performed on a Windows 10 64-bits PC equipped with an Intel(R) Core (TM) i5-7500 CPU @ 3.20GHz processor, and 8 GB-RAM. For deep learning technology, parallelizing calculation is an important power. Benefiting from GPU's parallelizing power, we used NVIDIA GTX 1060 GPU having 3GB of memory to reduce our training time. Also, we used high-level neural networks application programming interface of Tensorflow to implement our proposed deep learning model.

A. Classification based on the dataset S, R, S & R

In this section, we train the model on dataset with geometric transformations.

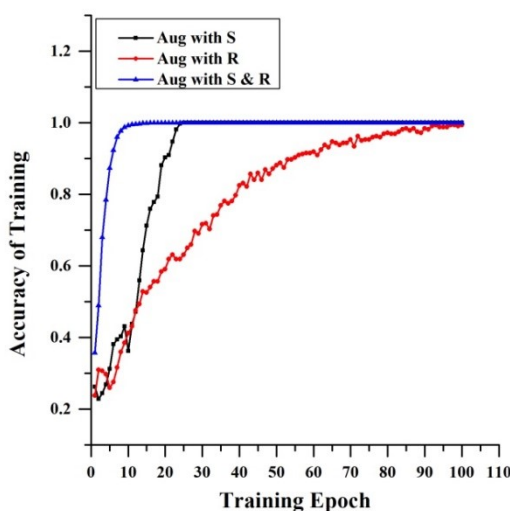


FIGURE 5. Iteration of training accuracies changes on dataset with augmented methods of geometric transformations.

Fig. 5 shows the curves of model accuracy rate changes as the number of iterations increases on S, R, and S & R datasets during training. From the results curve, we can find that the convergence of model on S & R is the fastest, followed by S. However, the convergence of model trained on R is the slowest and the accuracies of validation changes performance is not good as well (as shown in Fig. 6).

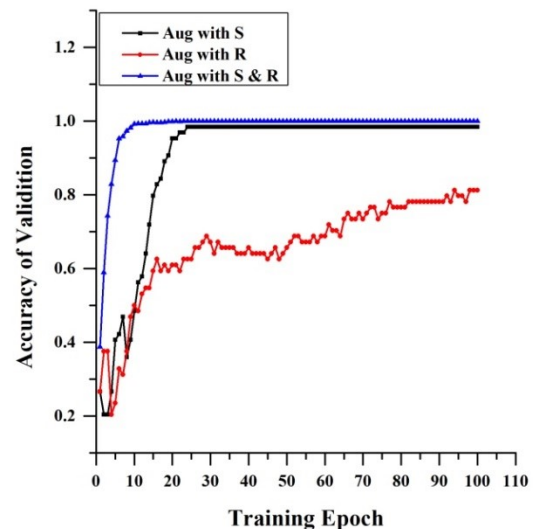


FIGURE 6. Iteration of validation accuracies changes on dataset with augmented methods of geometric transformations.

Then, we took the trained model to predict 100 pieces of untrained images. The results are presented in Table II. The best result is of the model trained on dataset S & R. It is worth mentioning that, although training on R did not perform very well, the prediction result is better than S.

TABLE II
PREDICTED RESULTS ON EACH CATEGORY OF THE MODEL
WHICH ARE TRAINED ON R, S AND R & S

Err rate	S	R	S & R
LV1(%)	2.70	0	0
LV2(%)	8.11	0	0
LV3(%)	2.70	5.41	0
LV4(%)	18.92	10.81	2.70
LV5(%)	13.51	16.22	8.11
Total(%)	45.94	32.44	10.81

B. Classification based on the dataset S with noise

In this section, we train the model on dataset S with three types of random noise. Fig. 7 shows the curves of accuracies of training changes as the number of iterations increases on S, S & PN, S & GN, S & SN.

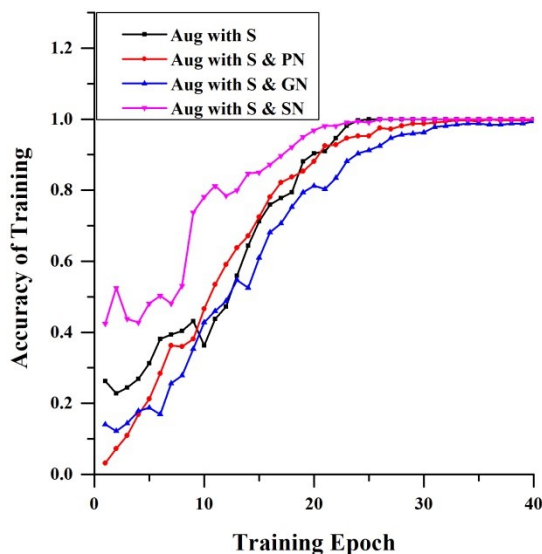


FIGURE 7. Iteration of training accuracies changes on S with noise of Pepper, Salt and Gauss separately.

From the results curves, we can find the models trained on all the datasets are fast to converge. After thirty epochs, all the models nearly complete to converge. Moreover, the accuracies of validation changes on these datasets are similar with the accuracies of training, and the accuracies of validation changes shown in **Fig. 8**. Beyond that, both the accuracy of training and validation reach at 96%.

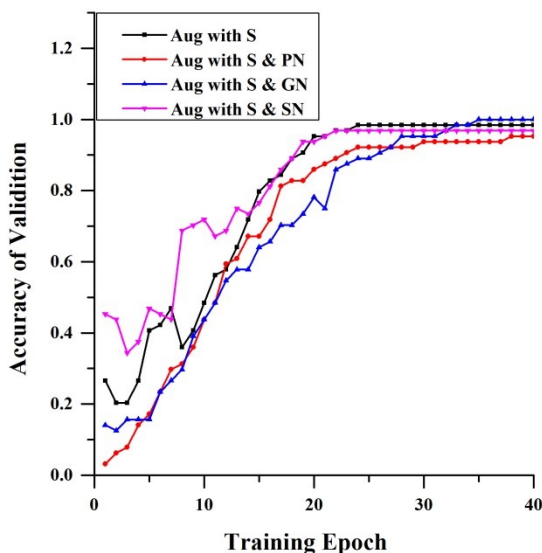


FIGURE 8. Iteration of validation accuracies changes on S with noise of Pepper, Salt and Gauss separately.

Then, we tested the predictive results of these models (see **Table III**). The table shows that the dataset of S with

Salt noise has the best effect on final results, followed by that with Gaussian noise and finally with Pepper noise.

TABLE III
PREDICTED RESULTS OF EACH CATEGORY OF THE MODEL TRAINED ON S WITH ADDED THREE KINDS OF NOISE

Err rate	S	S&PN	S & GN	S & SN
LV1(%)	2.70	5.41	0	0
LV2(%)	8.11	16.22	5.41	0
LV3(%)	2.70	2.70	8.11	8.11
LV4(%)	18.92	16.22	13.51	2.70
LV5(%)	13.51	10.81	13.51	10.81
Total(%)	45.94	51.36	40.54	21.62

C. Classification based on the dataset R with noise

In this section, we trained the model on dataset R with three types of random noise. **Fig. 9** shows the accuracies of training changes as the number of iterations increases on R, R & PN, R & GN, R & SN. Unlike the series of S, we can find the convergence rate of all these datasets much slower. Besides, the validation accuracy based on all these datasets is hard to promote it when reaches at nearly 80%, as shown in **Fig. 10**.

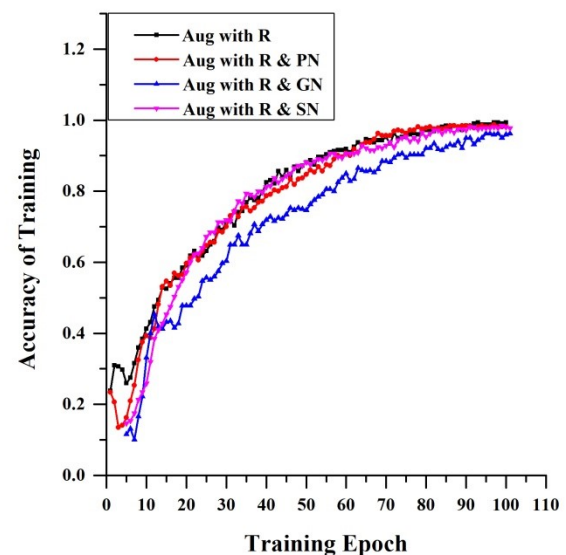


FIGURE 9. Iteration of training accuracies changes on R with noise of Pepper, Salt and Gauss separately.

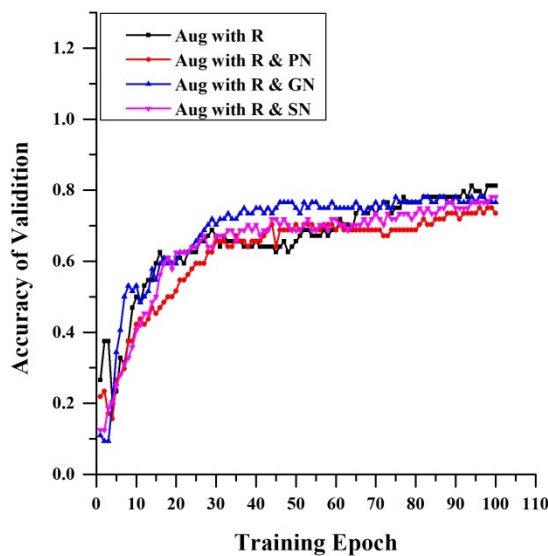


FIGURE 10. Iteration of validation accuracies changes on R with noise of Pepper, Salt and Gauss separately.

Although, training performances of these datasets are inferior to S series, they can be closely considered for the predicting results. The results are presented in **Table IV**.

TABLE IV
PREDICTED RESULTS OF EACH CATEGORY OF THE MODEL
TRAINED ON R WITH ADDED THREE KINDS OF NOISE

Err rate	R	R&PN	R & GN	R & SN
LV1(%)	0	0	0	0
LV2(%)	0	8.11	0	2.7
LV3(%)	5.41	2.70	0	5.41
LV4(%)	16.22	10.81	13.51	5.41
LV5(%)	10.81	13.51	10.81	8.11
Total(%)	32.44	35.13	24.32	21.63

D. Classification based on the dataset combined R & S

In this section, we train the model on dataset R & S with three types of random noise. **Fig. 11** shows the accuracies of training changes as the number of iterations increases on R & S, R & S & PN, R & S & GN, and R & S & SN with respect to these three datasets. **Fig. 12** shows the accuracies of validation changes during training on these datasets. From these curves we can find that the models trained under these datasets have the best performance both in convergence and accuracies changes. There were few shocks in training. Although training based on R series alone did not perform well, its performance improved when combined with S.

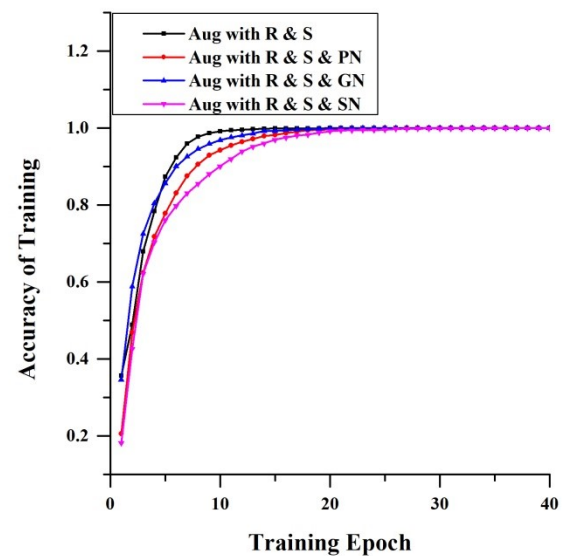


FIGURE 11. Iteration of training accuracies changes on R & S with noise of Pepper, Salt and Gauss separately.

This group also had better predictions than previous groups as obvious from the result shown in **Table V**. Similarly, the model trained on salt noise provides the best prediction results.

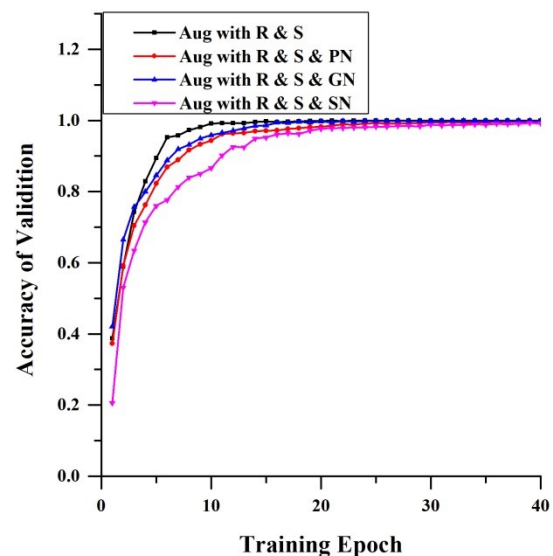


FIGURE 12. Iteration of validation accuracies changes on R & S with noise of Pepper, Salt and Gauss separately.

TABLE V
PREDICTED RESULTS OF EACH CATEGORY OF THE MODEL
TRAINED ON R & S WITH ADDED THREE KINDS OF NOISE

Err rate	R&S	R&S&PN	R&S & GN	R&S & SN
LV1(%)	0	0	0	0
LV2(%)	0	2.70	0	0
LV3(%)	0	0	5.41	2.70
LV4(%)	2.70	10.81	2.70	2.70
LV5(%)	8.11	18.92	2.70	2.70
Total(%)	10.81	32.43	10.81	8.1

E. Performance comparison with augmented methods

Image brightness and flip changes are two important methods of augmentation and solve the problem of over-fitting effectively. It is due to the images brightness as changes prevalently exist in natural environment. Therefore, we compared our method with these two augmentation methods in this study.

Firstly, we generated two datasets, one augmented by random brightness changes (BRIG), and the other augmented by flipping both horizontally and vertically (FLIP). Fig. 13 shows the accuracies of training changes as the number of iterations increases on BRIG, FLIP and R & S & SN.

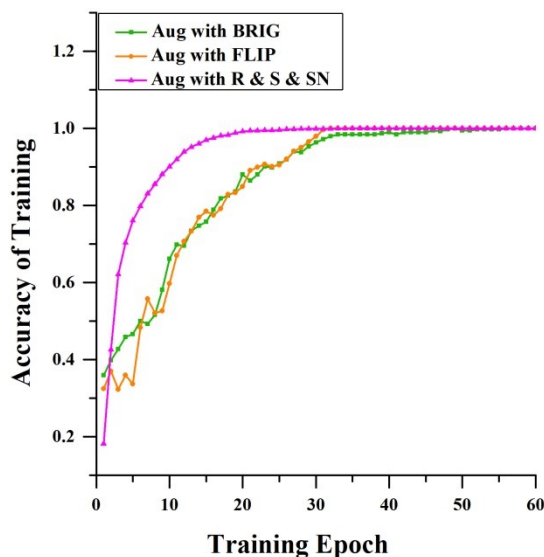


FIGURE 13. Iteration of training accuracies changes on the datasets with three augmentation methods.

Fig. 14 shows the accuracies of validation changes during training on these datasets. From these curves we can find the models trained on R & S & SN still have the best performance both on convergence and accuracies changes compared with the other two datasets.

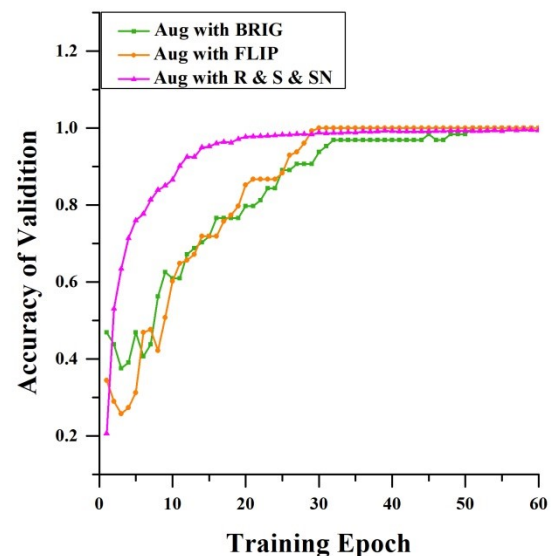


FIGURE 14. Iteration of validation accuracies changes on the datasets with three augmentation methods.

Again, we took these three trained models to predict 100 pieces of untrained images. The results are given in Table VI. The best results are obtained for the model trained on dataset R & S & SN.

TABLE VI
PREDICTED RESULTS ON EACH CATEGORY OF THE MODEL
TRAINED ON BRIG, FILP and R & S & SN

Err rate	BRIG	FILP	R&S & SN
LV1(%)	2.7	13.51	0
LV2(%)	18.92	8.11	0
LV3(%)	18.92	13.51	2.70
LV4(%)	8.11	16.22	2.70
LV5(%)	13.51	13.51	2.70
Total(%)	62.16	64.86	8.1

F. Runtime Analysis

Additionally, to promote the system, time cost plays an essential role. Therefore, we take response time of this system into consideration with two experimental conditions. One is under Intel(R) Core (TM) i5-7500 CPU, and the other is adding with NVIDIA GTX 1060 GPU. We tested the response time of our system under these two conditions to predict 100, 300, 500 and 700 pieces of images. The results are shown in Fig. 15.

The results demonstrated that the response time of our classification system was less than 1 millisecond (ms) per one hundred images, whether the device had parallel computing power or not.

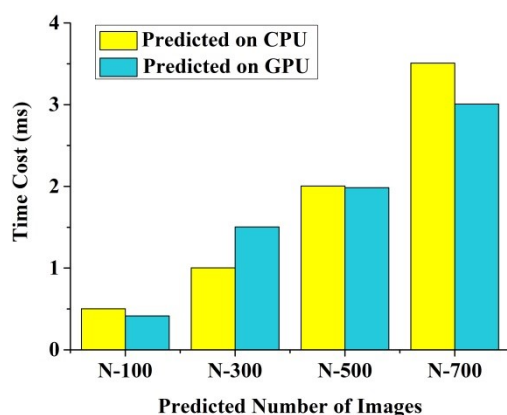


FIGURE 15. The time cost of the classification model on CPU or GPU condition for different number of images.

VI. DISCUSSION AND ANALYSIS

Recently, there are a number of studies on data augmentation methods, and most of these methods provide good performance in certain fields. From our research, we consider the characteristic of this task was found concurrent to meet object distance and angle changes in tomato classification. Therefore, adopting rotation, scale change and rotation with scale change are the three methods we used to augment our datasets.

Furthermore, Noise exists widely in natural environment. For approaching our dataset to the real environment condition, we add three types of noise, i.e. Pepper, Salt and Gaussian to the dataset. Adding Gaussian and Pepper noise brings colourful pixel to images. As the colourful information of images plays an important role in the classification task, such methods can create confusion. Comparatively, adding Salt noise will not introduce other colourful information except white, so the results would be much better compared with the other two methods.

VII. CONCLUSIONS

In this paper, we divide tomato into five categories according to different ripeness indices based on the relationship between the storage time and appearance. Here, we designed and implemented a novel architecture, based on deep learning for the classification of tomato maturity levels. Compared with other classical architectures, it has less parameter calculation and higher accuracy.

To achieve better performance of the designed classification model, we use t-SNE to verify the distribution of the dataset, and to delete bad images. In order avoid over-fitting problem during training of the model, we exploit three methods of augmentation for the datasets.

Through experiments on different groups of datasets, we obtained the best predicted results by training on the R & S & SN dataset. With this, the final accuracy reaches to 91.9% and the prediction time becomes less than 0.01 second per one hundred images.

REFERENCES

- [1] A. Sembiring, A. Budiman, and Y. D. Lestari, "Design and Control of Agricultural Robot for Tomato Plants Treatment and Harvesting," *J. Phys. Conf. Ser.*, vol. 930, no. 1, pp. 0–6, 2017.
- [2] L. L. Wang *et al.*, "Development of a tomato harvesting robot used in greenhouse," *Int. J. Agric. Biol. Eng.*, vol. 10, no. 4, pp. 140–149, 2017.
- [3] W. M. Syahrir, A. Suryanti, and C. Connysynn, "Color grading in tomato maturity estimator using image processing technique," *IEEE Int. Conf. Comput. Sci. Inf. Technol. (ICCSIT)*, pp. 276–280, 2009.
- [4] O. O. Arjenaki, P. A. Moghaddam, and A. M. Motlagh, "Online tomato sorting based on shape, maturity, size, and surface defects using machine vision," *Turkish J. Agric. For.*, vol. 37, no. 1, pp. 62–68, 2013.
- [5] Z. Ma, J. H. Xue, A. Leijon, Z. H. Tan, Z. Yang, and J. Guo, "Decorrelation of Neutral Vector Variables: Theory and Applications," *IEEE Trans. Neural Networks Learn. Syst.*, vol. 29, no. 1, pp. 129–143, 2018.
- [6] P. M. Pieczywek, J. Cybulska, A. Zdunek, and A. Kurenda, "Exponentially smoothed Fujii index for online imaging of biospeckle spatial activity," *Comput. Electron. Agric.*, vol. 142, pp. 70–78, 2017.
- [7] K. Jarrett, K. Kavukcuoglu, M. Ranzato, and Y. LeCun, "What is the best multi-stage architecture for object recognition?" *IEEE International Conference on Comput. Vision.*, pp. 2146–2153, 2009.
- [8] N. Goel and P. Sehgal, "Fuzzy classification of pre-harvest tomatoes for ripeness estimation - An approach based on automatic rule learning using decision tree," *Appl. Soft Comput. J.*, vol. 36, pp. 45–56, 2015.
- [9] V. Pavithra, R. Pounroja, and B. S. Bama, "Machine vision based automatic sorting of cherry tomatoes," *2nd Int. Conf. Electron. Commun. Syst. (ICECS)*, pp. 271–275, 2015.
- [10] H. Lu, F. Wang, X. Liu, and Y. Wu, "Rapid Assessment of Tomato Ripeness Using Visible/Near-Infrared Spectroscopy and Machine Vision," *Food Anal. Methods*, vol. 10, no. 6, pp. 1721–1726, 2017.
- [11] A. Mohapatra, S. Shanmugasundaram, and R. Malmathanraj, "Grading of ripening stages of red banana using dielectric properties changes and image processing approach," *Comput. Electron. Agric.*, vol. 143, no. 382, pp. 100–110, 2017.
- [12] T. Zhou, S. Yang, L. Wang, J. Yao, and G. Gui, "Improved Cross-Label Suppression Dictionary Learning for Face Recognition," *IEEE Access*, vol. PP, no. c, pp. 1–1, 2018.
- [13] J. Chen, Z. Liu, H. Wang, A. Nunez, and Z. Han, "Automatic Defect Detection of Fasteners on the Catenary Support Device Using Deep Convolutional Neural Network," *IEEE Trans. Instrum. Meas.*, vol. 67, no. 2, pp. 1–13, 2017.
- [14] J. Li, J. Feng, and C. C. J. Kuo, "Deep convolutional neural network for latent fingerprint enhancement," *Signal Process. Image Commun.*, vol. 60, no. June 2017, pp. 52–63, 2018.
- [15] Y. Li, X. Cheng, and G. Gui, "Co-Robust-ADMM-Net: Joint ADMM Framework and DNN for Robust Sparse Composite Regularization," *IEEE Access*, vol. PP, no. c, pp. 1–1, 2018.
- [16] Z. Ma, Y. Lai, W. B. Kleijn, Y. Z. Song, L. Wang, and J. Guo, "Variational Bayesian Learning for Dirichlet Process Mixture of Inverted Dirichlet Distributions in Non-Gaussian Image Feature Modeling," *IEEE Trans. Neural Networks Learn. Syst.*, vol. PP, pp. 1–15, 2018.
- [17] Y. Sun, X. Wang, and X. Tang, "Deep Learning Face Representation From Predicting 10 000 Classes," *CVPR*, pp. 1891–1898, 2014.
- [18] H. Huang, J. Yang, Y. Song, H. Huang, and G. Gui, "Deep Learning for Super-Resolution Channel Estimation and DOA Estimation based Massive MIMO System," *IEEE Trans. Veh. Technol.*, vol. PP, no. 1, p. 1, 2018.
- [19] G. Gui, H. Huang, Y. Song, and H. Sari, "Deep Learning for An Effective Non-Orthogonal Multiple Access Scheme," *IEEE Trans. Veh. Technol.*, vol. PP, no. c, p. 1, 2018.
- [20] Y. Li, J. Zhang, Z. Ma, and Y. Zhang, "Clustering Analysis in the Wireless Propagation Channel with a variational Gaussian Mixture Model," *IEEE Trans. Big Data*, vol. 7790, no. c, pp. 1–1, 2018.
- [21] H. Yu, Z. H. Tan, Z. Ma, R. Martin, and J. Guo, "Spoofing Detection in Automatic Speaker Verification Systems Using DNN Classifiers and Dynamic Acoustic Features," *IEEE Trans. Neural Networks*

- Learn. Syst.*, pp. 1–12, 2017.
- [22] X. Sun, G. Gui, Y. Li, R. P. Liu, and Y. An, “ResInNet: A Novel Deep Neural Network with Feature Re-use for Internet of Things,” *IEEE Internet Things J.*, vol. PP, no. FEBRUARY, p. 1, 2018.
 - [23] M. Brahimi, K. Boukhalfa, and A. Moussaoui, “Deep Learning for Tomato Diseases: Classification and Symptoms Visualization,” *Appl. Artif. Intell.*, vol. 31, no. 4, pp. 299–315, 2017.
 - [24] A. Fuentes, S. Yoon, S. C. Kim, and D. S. Park, “A robust deep-learning-based detector for real-time tomato plant diseases and pests recognition,” *Sensors (Basel)*, vol. 17, no. 9, pp. 1–10, 2017.
 - [25] S. Bargoti and J. Underwood, “Deep fruit detection in orchards,” *IEEE Int. Conf. Robot. Autom.*, pp. 3626–3633, 2017.
 - [26] S. Bargoti and J. P. Underwood, “Image Segmentation for Fruit Detection and Yield Estimation in Apple Orchards,” *J. F. Robot.*, vol. 34, no. 6, pp. 1039–1060, 2017.
 - [27] S. H. Wang, Y. D. Lv, Y. Sui, S. Liu, S. J. Wang, and Y. D. Zhang, “Alcoholism Detection by Data Augmentation and Convolutional Neural Network with Stochastic Pooling,” *J. Med. Syst.*, vol. 42, no. 1, 2018.
 - [28] J. Muñoz-Bulnes, C. Fernández, I. Parra, D. Fernández-Llorca, and M. A. Sotelo, “Deep fully convolutional networks with random data augmentation for enhanced generalization in road detection,” *2017 IEEE 20th Int. Conf. Intell. Transp. Syst.*, pp. 366–371, 2017.
 - [29] S. Hussein, R. Gillies, K. Cao, Q. Song, and U. Bagci, “TumorNet: Lung nodule characterization using multi-view Convolutional Neural Network with Gaussian Process,” *Int. Symp. Biomed. Imaging*, pp. 1007–1010, 2017.
 - [30] Z. Ma, A. E. Teschendorff, A. Leijon, Y. Qiao, H. Zhang, and J. Guo, “Variational bayesian matrix factorization for bounded support data,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 4, pp. 876–889, 2015.
 - [31] L. J. P. Van Der Maaten and G. E. Hinton, “Visualizing high-dimensional data using t-sne,” *J. Mach. Learn. Res.*, vol. 9, pp. 2579–2605, 2008.
 - [32] M. A. Hearst, S. T. Dumais, E. Osuna, J. Platt, and B. Schölkopf, “Support vector machines,” *IEEE Intell. Syst. their Appl.*, vol. 13, pp. 18–28, 1998.
 - [33] C. Tomasi, “Histograms of Oriented Gradients,” *Comput. Vis. Sampl.*, pp. 1–6, 2012.
 - [34] T. Lindeberg, “Scale Invariant Feature Transform,” *Scholarpedia*, vol. 7, no. 5, p. 10491, 2012.
 - [35] V. Nair and G. E. Hinton, “Rectified Linear Units Improve Restricted Boltzmann Machines,” *Proc. 27th Int. Conf. Mach. Learn.*, no. 3, pp. 807–814, 2010.
 - [36] R. . Salakhutdinov and G. . Hinton, “Replicated softmax: An undirected topic model,” *Adv. Neural Inf. Process. Syst.*, pp. 1607–1614, 2009.
 - [37] X. Li et al., “Supervised latent Dirichlet allocation with a mixture of sparse softmax,” *Neurocomputing*, vol. 312, pp. 324–335, 2018.
 - [38] P. T. De Boer, D. P. Kroese, S. Mannor, and R. Y. Rubinstein, “A tutorial on the cross-entropy method,” *Ann. Oper. Res.*, vol. 134, no. 1, pp. 19–67, 2005.
 - [39] C. H. Li and P. K. S. Tam, “An iterative algorithm for minimum cross entropy thresholding,” *Pattern Recognit. Lett.*, vol. 19, no. 8, pp. 771–776, 1998.